HACKWAVE 3.0 / OFFICIAL REVEAL

STATEMENTS

FIVE CHALLENGES. ONE BIG BUILD.

Five thoughtfully designed problem statements for teams ready to question the ordinary, connect ideas, and build what comes next.

Discover the problem. Think deeply. Build boldly.

01 — 05 / BUILD • TEST • ADAPT • CONNECT

PS 1 / SOFTWARE ENGINEERING

Self-Healing Software Tests

Intent-aware end-to-end testing that survives interface changes

THE PROBLEM

Traditional end-to-end tests are brittle. A small UI change—renaming a button, moving a component, or redesigning a page—can break a test even when the user workflow still works. Teams spend time repairing selectors instead of checking whether their product behaves correctly.

THE CHALLENGE

Build a testing system that understands the intent of a test rather than depending on fixed selectors. A test should describe the user's goal, interpret the current interface, and find the actions needed to complete the workflow.

EXAMPLE

Traditional test: `click("#checkout-button")`

Intent-based test: "Complete checkout with a valid credit card."

EXPECTED BEHAVIOUR

When buttons are renamed, elements move, or the checkout flow is redesigned, the system should detect that the old steps no longer fit, reconstruct the workflow, generate updated actions, and verify the intended outcome. It must distinguish harmless UI changes from genuine application failures.

DEMO REQUIREMENT

Run a test successfully on Version A. Intentionally change the interface to create Version B. Show the system detecting the change, explaining what broke, rebuilding the steps from the original intent, and successfully running the updated test.

PS 2 / AGENTIC WORKFLOWS

Autonomous RevOps Agent

Connect scattered customer signals to explain and act on revenue risk

THE PROBLEM

Sales teams keep customer information across CRM records, emails, calendars, product usage, and customer conversations. Since these signals are scattered, early signs of churn or expansion opportunities can be missed until it is too late.

THE CHALLENGE

Build an agent that connects signals across these sources, identifies revenue risks and opportunities, explains the evidence behind its conclusions, and recommends the next useful action.

EXAMPLE: A CUSTOMER AT RISK

Last meeting: 21 days ago • No next step recorded
Customer champion has not replied • Product usage is declining
Renewal is in 43 days

FROM ALERT TO REASONING

Instead of merely saying “Acme is at risk,” the agent should explain: “Acme is showing multiple churn signals. The renewal is approaching, product usage is declining, and there has been no follow-up since the last meeting.” It must connect evidence rather than produce an unsupported warning.

EXPECTED WORKFLOW

Identify the issue Explain the evidence Create a follow-up task Draft an email Update CRM Notify the account owner.

IMPLEMENTATION OPTIONS

Teams do not need to build every integration from scratch. Existing MCP servers or platforms such as HubSpot, Salesforce, Linear, Slack, Gmail, and Composio can connect the agent to relevant data and actions.

SAFETY REQUIREMENT

Human approval is required before consequential actions are carried out. Make proposed changes reviewable and keep a clear record of why the agent recommended them.

PS 3 / RESILIENT INFRASTRUCTURE

The Internet After the API

Help software survive undocumented changes in external services

THE PROBLEM

Modern software relies on APIs, SDKs, MCP servers, webhooks, and third-party services. If a dependency changes behaviour, disappears, becomes rate-limited, or returns subtly different data, applications can silently fail.

THE CHALLENGE

Build a system that observes external dependencies, learns their real behaviour, detects meaningful changes, assesses which parts of an application are affected, and automatically adapts—or proposes a safe migration.

SYSTEM PIPELINE

External service Observed behaviour Contract / behavioural model Change detection Impact analysis Adapter or migration Validation Updated system.

EXAMPLE: A SEMANTIC CHANGE

An endpoint previously returns `GET /users → name` but now returns `GET /users → display_name`. The system should identify which workflows depend on `name` and determine whether mapping the new field preserves the original meaning.

CORE REQUIREMENT

Do not rely only on documented schemas. Detect behavioural changes too: altered defaults, different error responses, changed pagination, rate limits, or data that remains valid JSON but changes meaning.

DEMO REQUIREMENT

Provide an application dependent on several simulated external services. Introduce undocumented changes, then demonstrate automatic detection, impact analysis, adaptation, and validation that affected workflows work again.

WHAT A STRONG SOLUTION SHOWS

Accurate change detection, dependency mapping, semantics-aware compatibility fixes, regression validation, and safe handling when automatic repair is uncertain.

CORE CHALLENGE

Build infrastructure that allows software to survive changes in systems it does not control.

PS 4 / HUMAN-CENTERED AI

Intent-to-Interface: Adaptive AI Workspace

An interface that reshapes itself around the user's current goal

THE PROBLEM

Modern applications make users navigate fixed dashboards, menus, filters, and workflows—even when their goals differ. Finding the right information can take more effort than completing the task itself.

THE CHALLENGE

Build an AI-powered adaptive workspace that understands a user's current intent and dynamically arranges the interface around the task they are trying to accomplish.

EXAMPLE: PREPARE FOR A CLIENT MEETING

The user says: "I need to prepare tomorrow's client meeting." The workspace could bring together a client overview, recent communication, open issues, product usage, the previous meeting, suggested talking points, and relevant actions.

EXPECTED CAPABILITIES

- Understand user intent and clarify ambiguity when needed
- Select relevant information and tools
- Dynamically arrange UI components for the task
- Generate task-specific interfaces while preserving a consistent design system
- Adapt as the user's goal changes
- Explain why particular information and actions were surfaced

DEMO REQUIREMENT

Start with a generic dashboard. Give the system three different tasks—for example, prepare for a client meeting, investigate a drop in product usage, and review overdue project work. Show the interface reorganising itself for each task.

DESIGN EXPECTATIONS

The result should remain usable, coherent, and accessible. It should not simply generate a page of text: it should select and arrange interface components that actually support the task.

WHAT A STRONG SOLUTION SHOWS

Accurate intent detection, useful component selection, smooth adaptation, consistent visual design, and understandable explanations for interface changes.

CORE CHALLENGE

Build an interface that adapts to the user's goal instead of forcing the user to adapt to the interface.

PS 5 / AI • FINANCE • KNOWLEDGE GRAPHS

Autonomous Financial Event Market Impact Graph

Trace how financial events propagate across sectors and assets

THE PROBLEM

Financial events rarely affect a single company. A geopolitical event, commodity shock, interest-rate decision, or regulatory change can propagate through sectors and financial instruments. Many systems show news as isolated stories rather than connected chains of impact.

THE CHALLENGE

Build an AI system that converts unstructured financial information into a dynamic dependency graph, showing how an event may influence economic variables, sectors, and assets.

ILLUSTRATIVE IMPACT CHAIN

Crude oil rises 12% Inflation expectations increase Interest-rate expectations shift Banks and real estate may be affected Relevant market indices and assets respond.

EXPECTED CAPABILITIES

- Detect financial events from unstructured information
- Extract companies, commodities, sectors, indicators, and instruments
- Identify and score relationships between entities
- Construct and update a knowledge graph
- Estimate plausible propagation paths and affected assets
- Incorporate time-series data where available
- Explain each impact chain and the evidence behind it

TECHNICAL CHALLENGE

Combine NLP, event extraction, knowledge graphs, time-series data, graph algorithms, and LLM-based reasoning. Distinguish observed facts from inferred relationships, and communicate uncertainty rather than presenting predictions as guarantees.

DEMO EXPECTATION

Introduce a financial news event and show the system extracting entities, building a connected graph, tracing multiple propagation paths, identifying potentially affected assets, and explaining why each connection exists.